

Team ESCOM-IPN at CLEF 2026: Multimodal Financial Decision Making using LLMs, Chronos, and Auto-Reflective Prompt Routing

Rodrigo¹, Jona¹, Baru¹, and David¹

¹Escuela Superior de Cómputo (ESCOM) - Instituto Politécnico Nacional (IPN),
Mexico City, Mexico

Abstract

This Working Note describes the participation of the ESCOM-IPN team in Task 3 of the CLEF 2026 FinMMEval lab. We propose a highly structured, multimodal retrieval-augmented generation (RAG) pipeline to predict financial trading actions (BUY, HOLD, SELL) across diverse assets. Our system integrates unstructured news and SEC filings with numerical scalars and temporal price sequences. We introduce a novel dual-strategy *Prompt-Builder* featuring a tripartite temporal memory (rolling buffer, self-reflection, and top-K retrieval), allowing Large Language Models (LLMs) to reason over current market contexts while continuously learning from empirical trading results. Furthermore, we implement *finvert*, a highly optimized FinBERT-based module for robust market sentiment extraction. Our methodology demonstrates how programmatic prompt engineering, heuristic risk management, and chronological FIFO queuing of semantic summaries can significantly enhance LLM reasoning in simulated financial environments.

Keywords: Financial Forecasting, Large Language Models, Sentiment Analysis, Prompt Engineering, Temporal Memory, FinMMEval, CLEF 2026.

1 Introduction

Financial decision-making requires the rapid synthesis of heterogeneous data sources. In the CLEF 2026 FinMMEval Lab (Task 3), participants are challenged to develop agents capable of processing numerical scalars, temporal price histories, and textual news to recommend trading actions (BUY, HOLD, SELL) for assets such as Bitcoin, Tesla, and Apple.

While Large Language Models (LLMs) excel at natural language understanding, they traditionally struggle with raw numerical sequences and lack inherent contextual memory of past actions in simulated trading environments. To bridge this gap, our team developed a comprehensive multimodal pipeline.

The core contribution of our work lies in the intelligent orchestration of data prior to LLM inference. Specifically, we detail two critical components designed by our team: (1) **finvert**, an automated and resource-optimized sentiment extraction module leveraging FinBERT, and (2) the **PromptBuilder**, a sophisticated prompt engineering engine equipped with a tripartite temporal memory. By providing the LLM not just with current data, but with a rolling history of its past decisions and their resulting financial rewards, we simulate a realistic trading agent that adapts to market dynamics and mitigates risk through self-reflection.

2 Related Work

What mystery does the market conceal when algorithms attempt to unravel its pulse? Forged at the intersection of natural language processing and financial volatility, the state of the art reveals an ecosystem where semantics and chaos converge.

Conceived to capture the heartbeat of the markets, recent literature exhibits a notable evolution in sentiment extraction. From subtle lexical oscillations to notable semantic deviations, foundational models such as FinBERT [1] and the FinGPT family [15, 17] have anticipated patterns that previously remained hidden in plain text.

Although it may seem straightforward, and despite the apparent linearity of numerical data, multimodal integration—surprisingly robust—demands rigorous evaluation frameworks. Driven by this necessity, the FinMultiTime dataset [14] and the daily repositories of the FinMMEval arena [9] unravel the convergence among news feeds, tabular data, and time series. Isolated in their own valley of complexity, classical theoretical approaches continue to illuminate the path; the estimation of mutual information [4] endures as an inescapable echo for measuring the dependence between these heterogeneous variables.

Structured under the paradigm of autonomous agents, modern trading systems have abandoned static decision-making to embrace dynamic workflows. With each test, with each adjustment, with each iteration, architectures like FINMEM [16] and TradingAgents [12] structure hierarchical memories and cognitive profiles. Precision and chaos coexist within the same algorithm: recent systems—such as TradingGroup [10], P1GPT [5], and frameworks evaluated in live environments [6]—articulate multifaceted reflections where diverse agents negotiate, debate, and converge upon a single market action.

Subjected to the relentless demand of continuous adaptation, foundational models have turned to advanced optimization techniques. Low-rank adaptations (LoRA) [3] and proximal policy optimization (PPO) [7] have enabled systems like Trading-R1 [11] and FLAG-TRADER [13] to align their predictions with direct financial rewards. On this very frontier, pure reasoning engines—embodied in architectures like DeepSeekMath [8] and DeepSeek-R1 [2]—anticipate a future where complex inference dominates over mere classification.

Ultimately, the predictive machinery and the unpredictable heartbeat of the market converge. The tools and frameworks presented here do not merely analyze the past; they sculpt the intelligence required to unravel the pulse of events yet to come.

3 Methodology

3.1 System Architecture and Dataset Formulation

[Notau: Expandir esta sección. Describir script 'generate_all.py', cómo se construyó 'finmmeval_raw.csv' mezclando HuggingFace y Yahoo Finance, y los detalles de Chronos Encoder.]

The overall architecture transforms raw multimodal data into structured prompts for LLM inference. The pipeline operates as follows:

1. **Data Ingestion:** Raw data from CLEF and Yahoo Finance is collected and unified, encompassing textual news, 10-K/10-Q filings, and technical scalars.
2. **Temporal Encoding (Chronos):** Time-series data is processed to extract temporal embeddings. *[David: Add Chronos details here].*
3. **Sentiment Extraction (finvert):** Textual data is classified to extract sentiment probabilities.
4. **Prompt Orchestration:** The *PromptBuilder* fuses all signals alongside the agent’s trading memory.

5. **LLM Decision Making:** The final prompt is submitted to the decision LLM to output a structured JSON containing the action and rationale.

3.2 Dataset and Feature Engineering

3.2.1 Data sources

The primary source is the *TheFinAI/daily_news* dataset [9], publicly available through HuggingFace, comprising daily financial records for twelve assets over approximately 268 trading days, from August 1, 2025 to April 25, 2026. The covered assets are ten listed equities (TSLA, MSFT, AAPL, AMZN, GOOGL, META, NVDA, ADBE, BMRN, and MRNA) and two cryptocurrencies (BTC and ETH).

Each daily record contains the closing price, a news summary of between 800 and 1500 words, SEC regulatory filings (forms 10-K and 10-Q) when available, a momentum label (bullish, neutral, or bearish), and the next-day price difference — used exclusively as a reward signal during training and never exposed to the model as an input.

As a complementary source, historical prices from Yahoo Finance were used for a four-month warm-up period prior to the start of the dataset (April–July 2025). Their sole purpose was to ensure that indicators with long lookback windows — the 60-day moving average regime signal and the 30-day rolling Sharpe ratio — could be computed without producing missing values at the start of the main period. These data were not incorporated as training rows, as doing so would have created an imbalance across assets since only TSLA and BTC had extended historical data of this type.

3.2.2 Feature engineering

A total of 43 candidate features were constructed across six categories. The common design criterion is the absence of temporal information leakage: every feature at time t uses only information available up to that point.

Returns and trend. Log returns are computed at one, three, and five days. The price position relative to its 10-, 20-, and 50-day moving averages is expressed as a percentage deviation. A binary market regime signal takes value +1 when the price exceeds the 60-day moving average (bullish regime) and −1 otherwise.

Volatility. Annualized realized volatility is computed over windows of 5, 10, 20, and 30 days. Two volatility ratios — $\text{vol}_{5d}/\text{vol}_{20d}$ and $\text{vol}_{10d}/\text{vol}_{30d}$ — detect the expansion or contraction of short-term volatility relative to longer baseline levels, a signal particularly relevant for cryptocurrencies. A binary volatility regime flag activates when the 10-day volatility exceeds the historical 70th percentile.

Technical oscillators and MACD. The RSI is computed at periods of 9 and 14 days and normalized to $[0, 1]$. Bollinger Bands contribute the percentage position of the price within the bands and the relative band width. MACD is decomposed into line, signal, and histogram components; the histogram is particularly useful for detecting imminent trend crossovers.

Agent internal state. The rolling Sharpe ratio at 30 and 10 days reflects the risk-adjusted performance of recent decisions. State confidence accumulates consecutive trading days without a significant price shock (return exceeding 3σ), normalized to $[0, 1]$, following the FINMEM framework [16].

Text-derived features. Sentiment analysis is performed with ProsusAI/FinBERT [1], a model specialized in financial texts. Since news summaries frequently exceed the 512-token limit, each document is split into 450-token chunks. Scores are averaged across chunks with a weight of $1.5\times$ applied to the first, as the introductory paragraph tends to concentrate the dominant sentiment signal. For regulatory filings (10-K and 10-Q), the same chunked procedure is applied; since filings appear at most once per quarter, the computed score is propagated forward using last observation carried forward (LOCF), and the number of days since the last available filing — normalized by 90 — is included to model the temporal decay of filing relevance.

3.2.3 Feature selection

Selection was performed jointly across all twelve assets (approximately 3 200 observations), providing more reliable mutual information estimates than per-asset analysis. The process consists of three sequential steps.

(1) Spearman correlation filter. Feature pairs with an absolute Spearman rank correlation coefficient exceeding 0.85 were identified. Spearman was preferred over Pearson because financial features exhibit asymmetric distributions and non-linear relationships. From each highly correlated pair, the feature with lower mutual information with respect to the target was discarded.

(2) Mutual information filter. Mutual information between each feature and the binary target — next-day price direction — was estimated using the k -nearest neighbors method with $k = 5$ [4]. Features scoring below 0.005 were discarded.

(3) Temporal stability. A two-sample Kolmogorov-Smirnov test was applied between the feature distributions of the first and last temporal folds. Features with KS statistic exceeding 0.30 were flagged for review, as distributional drift in financial series may reflect genuine structural regime changes rather than data quality issues. The analysis was run independently for equities and cryptocurrencies.

3.2.4 Final feature sets

After selection, 16 features were retained for equities and 16 for cryptocurrencies. Tables 1 and 2 detail the final sets. Both share a core of ten common features — returns, news sentiment, encoded momentum, and basic metadata — and differ in the technical signals with the highest predictive power for each asset class.

Feature	Range	MI	Rationale
asset_type	{0, 1}	—	Asset class identifier
day_of_week	[0, 6]	—	Calendar effects
month	[1, 12]	—	Monthly seasonality
has_news	{0, 1}	—	News availability
has_momentum	{0, 0.5, 1}	—	Momentum data quality
finbert_news_full	[−1, 1]	—	Sentiment (full text)
finbert_news_intro	[−1, 1]	—	Sentiment (intro paragraph)
momentum_encoded	{−1, 0, 1}	—	Momentum direction
log_return_1d	[−0.5, 0.5]	—	Base directional signal
log_return_3d	[−0.8, 0.8]	—	Short-term trend
state_confidence	[0, 1]	Med.	Context reliability
volatility_20d	[0, 3]	Med.	Asset risk level
rsi_9	[0, 1]	High	Overbought/oversold (fast)
rsi_14	[0, 1]	High	Overbought/oversold (standard)
news_volume_proxy	[0, 1]	Med.	Market attention
momentum_strength	[0, 5]	Med.	Vol.-adjusted movement strength

Table 1: Final selected features — Equities.

Feature	Range	MI	Rationale
asset_type	{0, 1}	—	Asset class identifier
day_of_week	[0, 6]	—	Calendar effects (incl. weekends)
month	[1, 12]	—	Monthly seasonality
has_news	{0, 1}	—	News availability
has_momentum	{0, 0.5, 1}	—	Momentum data quality
finbert_news_full	[−1, 1]	—	Sentiment (full text)
finbert_news_intro	[−1, 1]	—	Sentiment (intro paragraph)
momentum_encoded	{−1, 0, 1}	—	Momentum direction
log_return_1d	[−0.5, 0.5]	—	Base directional signal
log_return_3d	[−0.8, 0.8]	—	Short-term trend
sr_30d	[−3, 3]	High	Risk-adjusted recent performance
price_vs_ma_10	[−30, 30]%	High	Short-term relative position
rsi_14	[0, 1]	High	Overbought/oversold
vol_ratio_5_20	[0.3, 4.0]	Med.	Volatility expansion
state_confidence	[0, 1]	Med.	Context reliability
vol_ratio_10_30	[0.3, 4.0]	Med.	Volatility regime

Table 2: Final selected features — Cryptocurrencies.

3.2.5 Text treatment

News summaries and regulatory filings are stored separately from the numerical features and passed directly to the LLM as natural language. This decision rests on three observations.

First, FinBERT sentiment scores obtained near-zero mutual information values in both asset classes. The dataset’s texts are pre-processed summaries that frequently contain sentiment-neutralizing language — phrases such as “*overall market sentiment remains neutral*” cause FinBERT to underestimate the actual directional signal. Compressing 800–1 500 words into a single scalar discards the causal and entity-specific information that gives the text its predictive value. Second, large language models are significantly more capable of extracting actionable

signals from structured financial prose than scalar sentiment classifiers [17]. Third, keeping the text separate preserves flexibility with respect to the target model’s context window.

FinBERT scores are nonetheless retained in the numerical feature set as supplementary signals, not as the primary sentiment input.

3.3 Temporal Encoding: The Chronos-Bolt Module

To capture the predictive signal embedded in historical price trajectories, our pipeline employs **Chronos-Bolt** (`amazon/chronos-bolt-small`), a transformer-based probabilistic forecasting model pre-trained on large collections of real-world time series. Unlike traditional statistical approaches, Chronos-Bolt learns a distribution over future values, enabling the extraction of uncertainty-aware forecasts directly from raw numerical sequences.

The module is loaded via the `BaseChronosPipeline` abstraction, which handles device placement automatically. When a CUDA-capable GPU is available, the model is instantiated in `bfloat16` precision to reduce VRAM consumption while preserving numerical stability. In CPU-only environments, standard floating-point inference is applied as a fallback.

3.3.1 Input Formulation and Batch Processing

Each asset’s historical prices are stored per-day as a serialized list in the `prices_sequence` column of the dataset. Prior to inference, each entry is deserialized into a typed list of floats, forming the context window fed to the model.

To maximize GPU throughput, sequences are grouped into batches of 16 and converted to PyTorch tensors. For each batch, the model is queried with `prediction_length=1`, restricting the forecast horizon to a single future time step — the next trading day. This deliberate constraint enforces that the model can only reason over observed history, preserving temporal isolation and avoiding any form of look-ahead bias.

3.3.2 Forecast Extraction and Direction Classification

Chronos-Bolt outputs a probabilistic forecast tensor of shape $(\text{batch} \times \text{num_samples} \times \text{prediction_length})$, where the `num_samples` dimension represents draws from the learned predictive distribution. To obtain a single point estimate per sequence, we compute the **median** across the sample dimension:

$$\hat{p}_{t+1} = \text{median}_s F_s(x_{1:t}) \quad (1)$$

where F_s denotes the s -th sample of the forecast distribution conditioned on the observed price history $x_{1:t}$. The resulting scalar $\hat{p}_{t+1}$ is then converted into a discrete directional class by computing the relative price change with respect to the last observed price p_t :

$$\Delta = \frac{\hat{p}_{t+1} - p_t}{p_t} \quad (2)$$

A symmetric threshold $\tau = 0.001$ ($\pm 0.1\%$) governs the classification into three mutually exclusive categories:

$$\text{class} = \begin{cases} 1 & \text{if } \Delta > \tau \quad (\text{UP}) \\ 2 & \text{if } \Delta < -\tau \quad (\text{DOWN}) \\ 0 & \text{otherwise} \quad (\text{HOLD}) \end{cases} \quad (3)$$

This threshold was selected to filter out negligible price fluctuations below the level of market noise, ensuring that the HOLD class captures genuine price stability rather than prediction imprecision.

3.3.3 Evaluation and Output

Upon completing inference over the full dataset, the module computes both global and per-asset evaluation metrics. Global performance is assessed via accuracy, macro-averaged F1, and weighted F1 scores, while per-asset metrics allow identification of assets whose temporal price patterns are more or less amenable to the model’s forecasting strategy. The complete predictions and metrics are persisted to `predictions.csv` and `metrics.csv` respectively, serving as structured inputs to subsequent stages of the pipeline.

3.4 Multimodal Sentiment Extraction: The *finvert* Module

The *finvert* module acts as the market sentiment sensor of our architecture. Its primary objective is to process unstructured financial texts and quantify them into actionable numerical probabilities.

3.4.1 Auto-Discovery and Text Combination

Financial sentiment is rarely derived from a single source. Our system aggregates three primary textual streams per day in a strict priority hierarchy: daily news, annual reports (10-K), and quarterly reports (10-Q).

To optimize computational resources, the module implements an auto-discovery and filtering mechanism (`has_real_text`). Datasets lacking actual textual data (e.g., pure price-action assets from Yahoo Finance flagged as `[NO.TEXT]`) are automatically bypassed. These rows are deterministically assigned a *neutral* label with a 1.0 confidence score without invoking the GPU, vastly improving pipeline efficiency and preventing unnecessary VRAM allocation.

3.4.2 FinBERT Inference and Batch Processing

For valid textual entries, we utilize *FinBERT* (ProsusAI/finbert), a BERT model pre-trained on large-scale financial corpora. Texts are processed in batches of 32 to maximize GPU throughput.

The module tokenizes the combined daily text (truncated to a maximum length of 512 tokens) and outputs logits through a 3-class classification head. Applying a softmax function, we extract the probabilities for each class:

$$P(C_i|x) = \frac{e^{z_i}}{\sum_{j=1}^3 e^{z_j}} \quad (4)$$

where $C \in \{\text{positive}, \text{negative}, \text{neutral}\}$. The output is stored in a structured format containing the `sentiment_label`, the absolute `sentiment_score`, and the exact probability distribution. This granular output prevents the downstream LLM from interpreting a weak positive signal with the same weight as a strong one.

3.5 Dynamic Prompt Routing and Temporal Memory

The *PromptBuilder* is the central nervous system of our pipeline, encompassing over 600 lines of orchestration logic. Its responsibility is to synthesize sentiment analysis, technical scalars, portfolio status, and temporal sequences into a highly structured prompt.

Crucially, to simulate a realistic trading environment and prevent look-ahead bias, we enforce **strict temporal isolation**: the prompt for day T is generated entirely *before* the portfolio memory is updated with the market outcomes of day T .

3.5.1 Tripartite Memory Architecture

At any time step t , the state of the market is captured by a `SensorOutputs` object. Meanwhile, the system maintains a `MemoryContext` of `DayRecord` objects. Inspired by recent advancements in LLM trading agents (such as FLAG-TRADER and FINMEM), we implemented three types of memory:

- **Rolling Buffer:** A sequential tracker of recent actions and rewards.
- **Self-Reflection:** Analysis of past errors and successes.
- **Top-K Retrieval:** Extraction of the most relevant historical trades.

3.5.2 Strategy V1: Programmatic Auto-Reflection

Strategy V1 is a deterministic, highly efficient prompt routing mechanism requiring no auxiliary API calls. It programmatically translates numerical scalars (e.g., RSI, portfolio exposure) and categorical data into natural language.

The primary innovation in V1 is the **Auto-Reflection Summary**. The module identifies the most successful and most detrimental trades within a historical window. The generated prompt presents the LLM with an ASCII-formatted rolling history table and a specific reflection block:

```
"CASOS EXITOSOS RECIENTES: 2024-01-15: Accion=BUY, Reward=+0.0234...  
CASOS NEGATIVOS RECIENTES: 2024-01-17: Accion=SELL, Reward=-0.0120..."
```

This structural constraint forces the LLM to ground its current decision by learning from empirical past mistakes, effectively mimicking human reinforcement learning.

3.5.3 Strategy V2: Dual-LLM Pipeline and Semantic FIFO Queuing

To further enhance reasoning, Strategy V2 employs a chained dual-LLM architecture, operating as follows:

1. **LLM-1 (Summarizer):** A lightweight model (e.g., GPT-4o-mini or Llama-3.2-3B) reads the raw data of past days and translates the events into a rich, narrative summary. To ensure deterministic testing during development, we also implemented a `StubClient` factory pattern.
2. **FIFO Queuing & Caching:** These narrative summaries are pushed into a chronological First-In-First-Out (FIFO) queue. To heavily optimize API costs and reduce latency, outputs from LLM-1 are cached directly in the `DayRecord.llm_summary` attribute.
3. **LLM-2 (Decision):** A heavier reasoning model (e.g., GPT-4o) receives the entire narrative FIFO queue alongside the current market context to make the final BUY/HOLD/SELL decision.

3.5.4 Heuristic Risk Management via System Prompts

Both strategies are governed by a strict *System Prompt* that encodes risk-management heuristics. For instance, the LLM is explicitly instructed to prefer SELL or HOLD if the portfolio exposure exceeds 80%, to avoid BUY recommendations if cash is depleted, and to flag high volatility coupled with negative sentiment as extreme risk.

3.6 Model Optimization and Training Setup

3.6.1 Problem Formulation and Main Objectives

The primary objective of this study is to democratize the application of Group Relative Policy Optimization (GRPO) [8, 2] for financial trading decisions under severe hardware constraints. While recent state-of-the-art systems such as Trading-R1 [11] have demonstrated remarkable performance by scaling GRPO on massive GPU clusters (e.g., $8 \times \text{H200}$ nodes) with chain-of-thought (CoT) reasoning windows spanning 6,000 to 8,000 tokens, such infrastructure requirements place institutional-grade RL fine-tuning far beyond the reach of independent researchers and small laboratories. Conversely, alternative frameworks such as FLAG-TRADER [13] employ classical Proximal Policy Optimization (PPO) [7] with single-token outputs (Buy, Hold, or Sell), sacrificing interpretability and explicit reasoning capabilities entirely. Our work addresses this dichotomy by designing a training pipeline that preserves structured, interpretable reasoning while operating within a strict 12 GB VRAM budget, thereby bridging the gap between resource-intensive frontier research and accessible, reproducible financial machine learning.

A critical secondary objective involves resolving two well-documented failure modes of GRPO in constrained environments: the zero-variance collapse caused by aggressive length penalties on truncated responses, and the Nash equilibrium of inaction where the policy collapses to a trivial HOLD strategy. These phenomena, observed during preliminary autopsies of failed training runs, motivated a complete redesign of the reward architecture and action extraction logic to ensure stable gradient flow and meaningful exploration in low-computation regimes.

3.6.2 Architecture and Training Configuration

Base Model and Quantization Strategy All experiments utilize `unsloth/Phi-4-mini-instruct` as the base language model, loaded in 4-bit NF4 quantization via the Unsloth optimization framework. This configuration reduces the model footprint to approximately 2.5 GB of VRAM, leaving sufficient memory for adapter gradients, optimizer states, and the group generation buffer required by GRPO. Low-Rank Adaptation (LoRA) [3] is applied to all linear projection layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`) with a rank of $r = 4$ and scaling parameter $\alpha = 16$. Notably, the effective LoRA dropout is hardcoded to 0.0 in the training wrapper.

Hybrid Two-Stage Training Pipeline Our approach adapts and downscales the SFT-plus-GRPO curriculum originally proposed by Trading-R1 [11]—which was designed for 141 GB server-grade deployments—to commodity hardware. The pipeline consists of two strictly sequential phases.

Phase 1: SFT Warm-up. Prior to any reinforcement learning, the model undergoes supervised fine-tuning (SFT) for exactly one epoch (approximately 30 minutes) over 1,082 training examples. The objective of this phase is not to learn trading strategy but to anchor the structural XML output format. Each training example is formatted using the `grpo_cot_v1` template, which requires the model to generate a response containing `<technicals>`, `<news>`, `<conclusion>` tags, and a final action token of the form `[[[ACTION]]]`. The SFT target completions are generated via a dedicated builder that produces ultra-concise ground-truth responses of approximately 60 to 80 characters. This brevity is essential because teaching the model to produce long outputs during SFT, only to truncate them during GRPO due to the 128-token generation limit, leads to catastrophic intra-group variance collapse. The SFT trainer is configured with `batch_size=1`, gradient accumulation over 32 steps, a learning rate of 1.155×10^{-5} (half the RL learning rate for stability), and `bf16` mixed precision with gradient checkpointing.

Phase 2: GRPO Optimization. Following the SFT phase, the base model is reloaded together with the SFT adapters, and GRPO training commences for one epoch (approximately

6 minutes) over the same training corpus. The GRPO configuration is detailed in Table 3. The group size is restricted to $G = 4$ (the minimum viable for GRPO’s relative advantage estimation), and the maximum completion length is capped at 128 tokens—a drastic compression compared to the thousands of tokens used in prior work. To stabilize learning with an effective batch size of 1, we employ gradient accumulation across 32 steps. The KL divergence coefficient is set to $\beta = 0.051$, and the clipping threshold is $\epsilon = 0.104$.

Table 3: GRPO Training Hyperparameters

Parameter	Value
Base Model	Phi-4-mini-instruct (4-bit NF4)
LoRA rank r	4
LoRA α	16
LoRA dropout	0.0
GRPO group size G	4
KL coefficient β	0.051
Clip ϵ	0.104
Max completion length	128 tokens
Temperature	0.573
Learning rate	2.31×10^{-5}
Per-device batch size	1
Gradient accumulation	32
Optimizer max grad norm	0.5
Epochs executed	1 (SFT) + 1 (GRPO)

Ground Truth Signal Generation The ground truth labels for supervised warm-up and reward computation are derived from a volatility-normalized momentum signal: $\text{signal} = \log_{\text{return_t1/rolling_std}}$. The classification thresholds, optimized via Optuna TPE with 3 trials, partition the distribution into Buy (top 35.82%), Sell (bottom 19.18%), and Hold (remaining 45%). The dataset is temporally filtered to the most recent 12 months (360 days), yielding 1,082 training rows and 896 validation rows spanning from 2024-02-07 to 2025-02-01. Only rows containing news are retained (`has_news = 1`), ensuring that each example presents a multimodal decision task combining technical indicators, sentiment, and market news.

3.6.3 Optuna Search and Experimental Protocol

The hyperparameter search was coordinated using Optuna with the TPE (Tree-structured Parzen Estimator) sampler and a fixed seed of 42 to ensure reproducibility across runs. Due to compute constraints, the initial experiments were executed in a fast mode with only 3 trials; each trial ran the full pipeline (dataset construction, SFT warm-up, and GRPO optimization) and dumped configuration metadata to a `metadata.json` file. This fast-mode configuration prioritized quick, reproducible validations to detect stability issues (e.g., collapse to HOLD) before committing larger computational budgets.

The search space included high-impact hyperparameters such as `learning_rate`, `kl.beta`, `clip_epsilon`, `temperature`, `reward_scale_factor`, and the ground-truth thresholds used to label BUY/SELL decisions. Optuna suggested candidate configurations that were evaluated using the validation fitness metric (median Sharpe Ratio penalized by Maximum Drawdown above 20%). While the automated search identified candidates with superior fitness (for example, a trial with `fitness=1.974` and `MDD=17.48%`), the final deployment decision favored a model selected manually based on absolute Cumulative Return (90.67%) together with operational considerations.

This protocol proved effective for pipeline debugging and for calibrating the asymmetric reward function, but it also highlights the need to increase the number of trials and the computational budget in future runs to prioritize risk-adjusted metrics during automated model selection. Due to limited time, we were unable to run long-form trainings; therefore, all reported Optuna results correspond to fast-mode trials that executed the full pipeline in a shortened configuration.

3.6.4 Reward Function Design and Engineering Contributions

Asymmetric Reward Matrix The core behavioral signal is provided by an asymmetric 3×3 reward matrix that penalizes directional errors according to their financial severity, scaled by a factor of 1.5799. Table 4 presents the exact values employed in our experiments.

Table 4: Asymmetric Reward Matrix (Values Before Scaling)

Pred. \ True	BUY	HOLD	SELL
BUY	+1.0	−1.0	−2.25
HOLD	− 2.0	+1.0	− 2.0
SELL	−2.0	−1.0	+1.0

Our key adaptation over the asymmetric matrix proposed by Trading-R1 [11] lies in the calibration of the false-HOLD penalties. In the original formulation, the penalty for incorrectly predicting HOLD during a market rally or crash was set to -1.0 (or at most -1.5). Through empirical autopsy of failed training runs, we computed the expected reward of a trivial HOLD policy on our specific data distribution—encompassing volatile crypto assets such as BTC, ETH, and SOL—and found it to be -0.0945 , which dominated the expected rewards of BUY (-0.3856) and SELL (-1.1040). This created a Nash equilibrium of inaction: the model learned that remaining passive incurred the least damage, collapsing to 100% HOLD predictions. By intensifying the false-HOLD penalty to -2.0 , we mathematically destroyed this equilibrium, equalizing the cost of inaction with that of catastrophic directional errors and thereby forcing the policy to explore meaningful trading decisions.

Decision Placement Reward and Length Penalty Drawing on the Decision Placement Reward and Length Penalty concepts introduced by Trading-R1 [11], we implemented a composite format reward and a progressive length penalty adapted to our 128-token generation ceiling. The format reward distinguishes three cases: (i) a complete response with properly closed action tags receives $+0.5$; (ii) a response containing a detectable action but truncated before closing receives $+0.25$; (iii) a severely truncated or malformed response receives -0.5 ; and (iv) a response with no recognizable action incurs the invalid-format penalty of -1.5 scaled by the reward factor (yielding approximately -2.37).

The length penalty operates progressively rather than abruptly. Given a soft target of 240 characters (approximately 60 tokens), the penalty is computed as $\max(0, \text{len}(\text{response}) - 240) \times 0.0025$. This design prevents the sudden truncation shocks that kill intra-group variance in GRPO when all four group members hit the hard token limit simultaneously.

Truncated Token Rescue A fully original engineering contribution is the robust action extractor capable of rescuing partially generated tokens when the model exhausts its 128-token budget mid-action. For instance, a response terminating in `[[[BU` is heuristically resolved to BUY, while `[[[SEL` resolves to SELL. This mechanism is not merely a post-processing convenience; it is a critical MLOps technique to prevent zero-variance gradient death. In standard GRPO, if all group completions are truncated identically before emitting a valid action token,

the relative advantage across the group becomes zero, and policy gradients vanish. By extracting semantic signal from incomplete patterns, we maintain non-zero variance and preserve learning signals throughout training on limited hardware.

Complete Reward Formula The total reward for each generated completion is computed as:

$$R = (M \times 1.5799) + F - L, \quad (5)$$

where M is the matrix reward from Table 4, F is the format reward, and L is the length penalty. This composite function balances directional accuracy, structural compliance, and generative conciseness within an ultra-short reasoning window.

3.7 Results and Validation

3.7.1 Validation Protocol

Validation is conducted under two distinct regimes to balance training throughput against statistical robustness. During training, a fast-mode evaluation assesses performance on the last temporal fold only, limited to 100 rows, providing rapid feedback for Optuna trial selection. After training completion, a full validation is executed over all 896 validation rows divided into three contiguous temporal folds: Fold 0 (2024-02-07 to 2024-06-05, 360 rows), Fold 1 (2024-06-06 to 2024-10-03, 294 rows), and Fold 2 (2024-10-04 to 2025-02-01, 242 rows). For each fold, actions are simulated with weights $w_t \in \{+1, 0, -1\}$ corresponding to BUY, HOLD, and SELL, and daily returns are computed as $r_t = w_t \times (P_t - P_{t-1})/P_{t-1}$. From these returns, we derive Cumulative Return (CR), Sharpe Ratio (SR), Maximum Drawdown (MDD), Accuracy, Win Rate, and Profit Factor. The aggregate fitness score is defined as:

$$\text{fitness} = \text{median_SR} - \max(0, \text{MDD_worst} - 0.20) \times 10, \quad (6)$$

which severely penalizes any drawdown exceeding the 20% Hall of Fame threshold.

Validation assets The validation sets used in these experiments contained only YAHOO-sourced assets: BTC_YAHOO, ETH_YAHOO and SOL_YAHOO. Fast-mode validation (used during training and Optuna trials) evaluated only the last temporal fold and a limited sample (100 rows) drawn from these YAHOO assets. Full validation likewise operates on the same YAHOO assets across the three temporal folds described above. Assets originating from the CLEF split (for example, BTC_CLEF, ETH_CLEF, TSLA_CLEF, MSFT_CLEF and BMRN_CLEF) were reserved exclusively for the out-of-sample test set and were not included in validation runs.

3.7.2 Training and Fast-Mode Results

Within the fast-mode evaluation, the winning trial (Trial 0, `exp_20260506_085649_254046`) achieved a fitness of 2.9488 with an MDD of merely 0.11% over 100 rows. The action distribution at this stage was extremely conservative: 96% HOLD, 4% SELL, and 0% BUY. While this distribution suggested the model had learned to avoid catastrophic errors, it also indicated that the anti-HOLD calibration had not yet fully activated exploratory BUY behavior within the restricted fast-mode window. The two subsequent Optuna trials (using Qwen2.5-3B and a different hyperparameter configuration for Phi-4-mini, respectively) collapsed to repetitive text or 100% HOLD, yielding negative fitness values and confirming the brittleness of GRPO under hardware constraints without the stabilizing modifications described in the previous section.

3.7.3 Full Validation Results

The complete validation stage evaluated 9 candidate models on the full validation set (896 rows across three temporal folds). The full-validation protocol assessed temporal robustness and risk-adjusted performance, prioritizing a fitness metric defined as the median Sharpe Ratio penalized by excessive Maximum Drawdown ($\text{MDD} > 20\%$).

Performance Metrics Each finalist’s performance is evaluated across seven complementary dimensions. The following table provides a complete description of each metric used in Table 6. The fitness metric serves as the primary optimization objective, while the remaining six metrics capture different aspects of trading efficacy: risk-adjusted profitability (SR), cumulative return (CR), capital preservation (MDD), directional accuracy (Acc.), trade success rate (WR), and cumulative profitability scaled by drawdown magnitude (PF).

Table 5: Performance Metrics Reference

Metric	Definition	Interpretation
Fitness	Median Sharpe Ratio with penalty multiplier $-2.0 \times \max(0, \text{MDD} - 0.20)$ for drawdowns exceeding 20%	Primary optimization objective; higher is better; severe penalty for exceeding risk threshold
SR (Sharpe)	Excess return divided by portfolio volatility	Risk-adjusted profitability; higher indicates stable returns relative to fluctuations
MDD	Maximum cumulative loss from peak to trough (%)	Key downside risk measure; values $>20\%$ trigger fitness penalties; lower is safer
CR (Cumulative Return)	Total percentage gain or loss across the entire validation period (%)	Absolute profitability over the full period; higher indicates greater total wealth accumulation; can be misleading without considering drawdown
Acc. (Accuracy)	Proportion of BUY/HOLD/SELL decisions aligned with realized price direction	Directional correctness of model predictions; higher indicates better signal quality
WR (Win Rate)	Percentage of closed trades generating profit	Trade success frequency independent of magnitude; higher indicates consistent small wins
PF (Profit Factor)	Cumulative gains divided by cumulative losses	Net profitability; >1.0 profitable, <1.0 net losses, Inf/undefined indicates no losses

The model 085649_254046, which serves the current inference endpoint, achieved a Cumulative Return of 90.67%, the highest among all candidates, alongside a Sharpe Ratio of 2.9117 and an accuracy of 38.31%. However, its Maximum Drawdown of 48.05% far exceeds the 20% safety threshold, resulting in a suppressed fitness score of 0.1069.

In sharp contrast, model 111504.33de19 attained a fitness of 1.974—18.5 times higher—with an MDD of 17.48%, comfortably within the Hall of Fame threshold. This model also exhibited a superior win rate (58.14%) and accuracy (41.85%), making it objectively superior under virtually all risk-adjusted criteria.

Table 6: Full Validation Results — 9 Finalists

Exp.	Fitness	SR	MDD	CR	Acc.	WR	PF
085649_254046	0.1069	2.9117	48.05%	90.67%	38.31%	54.83%	1.513
111504_33de19	1.9740	1.9740	17.48%	19.34%	41.85%	58.14%	1.687
113530_8bc775 (ep4)	1.2134	1.2134	14.15%	9.44%	40.96%	52.93%	1.697
004508_27f172	-0.0759	3.0408	51.17%	89.76%	38.87%	55.81%	1.568
115709_051aed	-0.4434	1.4039	38.47%	22.74%	38.99%	51.91%	1.249
014550_f9ad37	-0.8756	1.4105	42.86%	28.16%	40.23%	50.90%	1.225
113530_8bc775 (ep2)	-0.6082	-0.1113	24.97%	-1.12%	42.12%	51.23%	0.970
221904_9bb326	-2.5173	-0.2100	43.07%	-3.18%	43.64%	50.89%	0.928
204602_88701e	-1.4462	-0.1954	32.51%	-1.89%	43.72%	48.85%	0.912

3.7.4 Analysis and Model Selection Trade-off

The discrepancy between the highest-fitness model and the deployed model illustrates a fundamental tension in financial agent design: the conflict between risk-adjusted efficiency and absolute profitability. Model 111504_33de19 represents the prudent choice under classical portfolio theory, delivering a Sharpe Ratio of 1.974 with capital preservation (MDD<20%). It embodies the risk-averse profile advocated by systems such as TradingGroup [10] and FINMEM [16], which emphasize dynamic risk management and survival across market regimes. Conversely, model 085649_254046 sacrifices downside protection for asymmetric upside capture, yielding a 90.67% cumulative return at the cost of a 48.05% drawdown—a profile closer to high-conviction discretionary trading than to institutional risk constraints.

The final selection of 085649_254046 was a manual, deliberate decision based on the competition’s primary evaluation metric: Cumulative Return. While this choice acknowledges the model’s vulnerability to severe capital erosion—as evidenced by its failure to pass the automatic Hall of Fame criteria—it also reflects the reality that in zero-sum or tournament-style evaluation environments, raw alpha generation often outweighs Sharpe-based efficiency. The existence of 111504_33de19 demonstrates that our training pipeline is capable of producing both profiles, and that the selection criterion, not the learning algorithm, drives the ultimate risk-return posture of the deployed agent.

Furthermore, the validation results expose the dangers of relying solely on fast-mode metrics. The fast-mode MDD of 0.11% for the deployed model was catastrophically optimistic compared to the full-validation MDD of 48.05%, confirming that evaluation over a single truncated fold produces an illusion of robustness. This finding reinforces the necessity of multi-fold temporal validation in financial machine learning, where data non-stationarity and regime shifts can render point-in-time metrics misleading.

4 Conclusion

In this Working Note, we presented a comprehensive RAG pipeline for financial decision-making in the CLEF 2026 FinMMEval Lab. By developing the *finvert* module for precise sentiment extraction and a novel *PromptBuilder* equipped with a tripartite temporal memory, we successfully bridged the gap between raw quantitative market data and qualitative LLM reasoning.

Our dual-strategy prompt orchestration demonstrated that providing an LLM with a structured, self-reflective memory of its past actions significantly enhances its ability to manage risk and make context-aware trading decisions.

Future work will focus on *[agregar sus puntos sobre mejorar el split temporal (Walk-forward validation), balancear el dataset entre equity/crypto, y automatizar la selección por fitness en lugar de selección manual por CR.]*.

Acknowledgments

We extend our gratitude to the Artificial Intelligence Club (GAIA) at ESCOM-IPN for their continuous support and computational resources.

5 Declaration on Generative AI

References

- [1] Dogu Araci. “FinBERT: Financial Sentiment Analysis with Pre-trained Language Models”. In: *arXiv preprint arXiv:1908.10063* (2019). URL: <https://arxiv.org/abs/1908.10063>.
- [2] DeepSeek-AI et al. “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning”. In: *arXiv preprint arXiv:2501.12948* (2025).
- [3] Edward J Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2022.
- [4] Alexander Kraskov, Harald Stögbauer, and Peter Grassberger. “Estimating mutual information”. In: *Physical Review E* 69.6 (2004), p. 066138. DOI: 10.1103/PhysRevE.69.066138.
- [5] Chen-Che Lu, Yun-Cheng Chou, and Teng-Ruei Chen. “P1GPT: A Multi-Agent LLM Workflow Module for Multi-Modal Financial Information Analysis”. In: *arXiv preprint* (2025). Neurowatt AI.
- [6] Lingfei Qian et al. “When Agents Trade: Live Multi-Market Trading Benchmark for LLM Agents”. In: *arXiv preprint* (2025).
- [7] John Schulman et al. “Proximal policy optimization algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017).
- [8] Zhihong Shao et al. “DeepSeekMath: Pushing the limits of mathematical reasoning in open language models”. In: *arXiv preprint arXiv:2402.03300* (2024).
- [9] TheFinAI. *daily_news: Daily Financial News Dataset for FinMMEval*. https://huggingface.co/datasets/TheFinAI/daily_news. Accessed: 2025. 2025.
- [10] Feng Tian, Flora D. Salim, and Hao Xue. “TradingGroup: A Multi-Agent Trading System with Self-Reflection and Data-Synthesis”. In: *arXiv preprint* (2025).
- [11] Yijia Xiao et al. “Trading-R1: Financial Trading with LLM Reasoning via Reinforcement Learning”. In: *arXiv preprint* (2025). Tauric Research.
- [12] Yijia Xiao et al. “TradingAgents: Multi-agents LLM financial trading framework”. In: *arXiv preprint arXiv:2412.20138* (2024).
- [13] Guojun Xiong et al. “FLAG-TRADER: Fusion LLM-Agent with Gradient-based Reinforcement Learning for Financial Trading”. In: *arXiv preprint arXiv:2502.11433* (2025).
- [14] Wenyan Xu et al. “FinMultiTime: A Four-Modal Bilingual Dataset for Financial Time-Series Analysis”. In: *arXiv preprint* (2025).
- [15] Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. “FinGPT: Open-Source Financial Large Language Models”. In: *arXiv preprint arXiv:2306.06031* (2023).
- [16] Yangyang Yu et al. “FinMem: A Performance-Enhanced LLM Trading Agent with Layered Memory and Character Design”. In: *arXiv preprint arXiv:2311.13743* (2024). URL: <https://arxiv.org/abs/2311.13743>.

- [17] Boyu Zhang, Hongyang Yang, and Xiao-Yang Liu. “Instruct-FinGPT: Financial Sentiment Analysis by Instruction Tuning of General-Purpose Large Language Models”. In: *arXiv preprint arXiv:2306.12659* (2023). URL: <https://arxiv.org/abs/2306.12659>.